Notes :→

(*) Recursion or the brute force approach takes
→ Time → $O(2^n)$ → Space → $O(n)$ { Recursion stack }

(*) Memoisation approach takes :
→ Time → $O(n) \times O(n) = O(n^2)$
(n elements) (recursion stack)

→ Space → { $O(n)$ for the recursion stack ; n elements ← $O(n)$ for the dp array }

(*) Tabulation approach takes :
Time → $O(n)$ for loop (2 → n)
Space → $O(n)$ → dp (n+1)

(*) Space Optimisation → $O(1)$
→ Just Use Variables
n_1, n_2
TC → $O(n)$ (for loop) (n^{th})

3 vars int p2 = 0, int p1 = 1; int curr;

p2	p1	curr
0	1	×
1	1	1
1	2	2
2	3	3
3	5	5
5	8	8

```
for (2 to n) O(n) {
    curr = p2 + p1
    p2 = p1
    p1 = curr
}
```
S/C = O(1)

* Points to remember:

Recursion → We are forced to calculate all the values even though we may have the answer. (Very slow)

Memoisation → In the memoisation approach we don't calculate known values, we return it. We only calculate unknown values. (Slightly faster)

Tabulation :→ In the tabulation method we store the known & find the unknown by looping. (faster)

(Space) Space Optimisation : In this approach we see if we can move from $O(n)$ to $O(1)$ (This is not possible for all problems)

Introduction to 2D – DP :→

Longest Common Subsequence

$O(m \times n)$

A = | b | d | \0 | (0 1 2)

B = | a | b | c | d | \0 | (0 1 2 3 4)

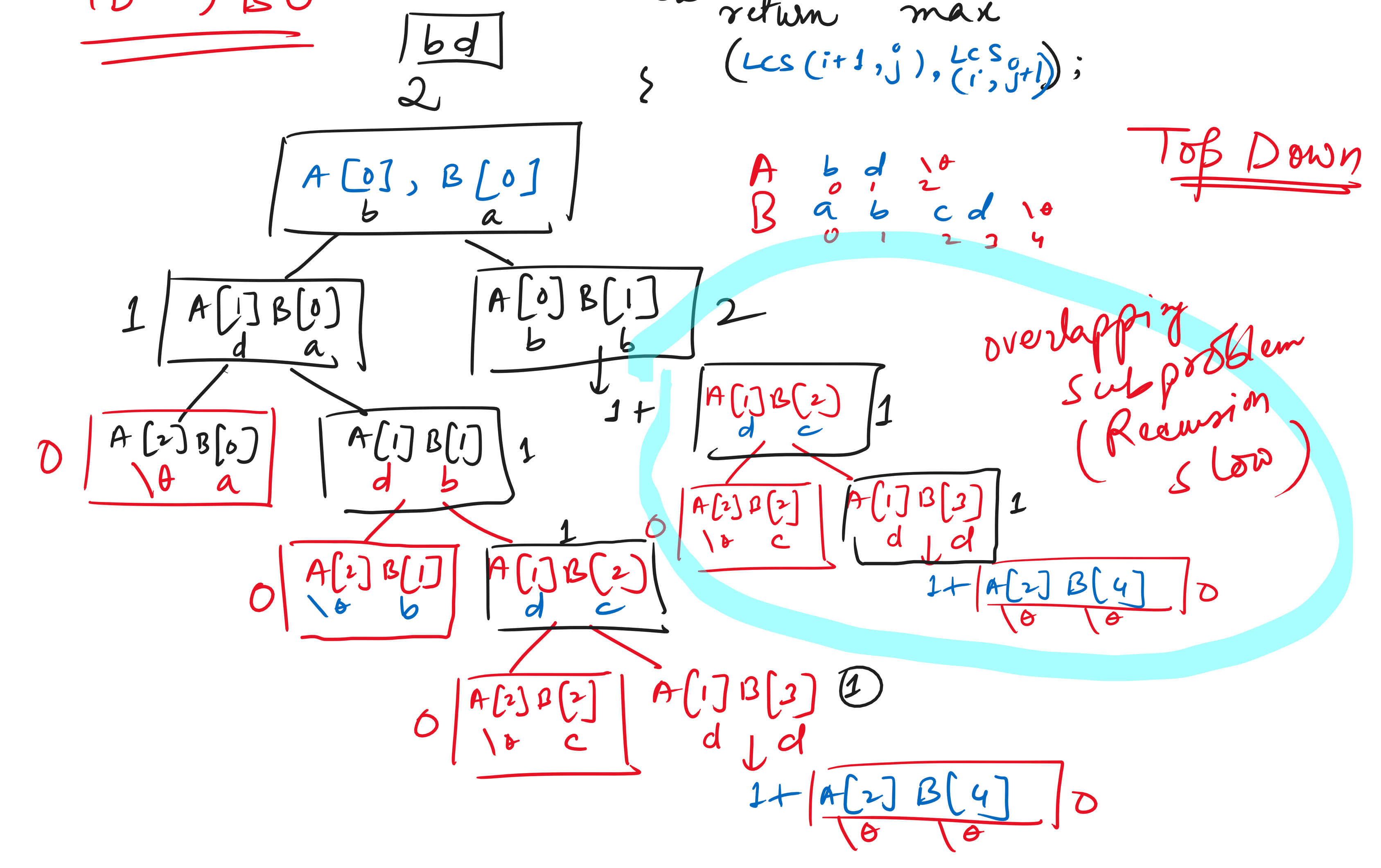
LCS = 2 (bd)

	a (0)	b (1)	c (2)	d (3)	\0 (4)
b 0	2	2			
d 1	1	1	1	1	
\0 2	0	0	0		0

TD → BU

[LCS] Pseudo Code

```
int LCS (i, j)
{
  if (A[i] == '\0' || B[j] == '\0')
      return 0;
  else if (A[i] == B[j])
      1 + LCS (i+1, j+1);
  else
      return max (LCS (i+1, j), LCS (i, j+1));
}
```



Dynamic Programming Approach :→

We initially fill the array with 0 to start.

A = | b | d | (1 2)

B = | a | b | c | d | (1 2 3 4)

bd = (2)

(Bottom-Up Approach)
Table fills top to bottom but O/p at bottom

	0	1 (a)	2 (b)	3 (c)	4 (d)
0	0	0	0	0	0
(b) 1	0	0	1	1	1
(d) 2	0	0	1	1	2

pseudo code:

```
if (A[i] == B[j])
    LCS (i,j) = 1 + A[i-1][j-1]      → 1 + prev diag
else
    LCS [i,j] = max(LCS [i-1, j], LCS [i, j-1])   → max of prev row, prev col
```

LCS Example 2 :→ Str1 = stone
Str2 = Longest / (One) (3)

	0	1 (l)	2 (o)	3 (n)	4 (g)	5 (e)	6 (s)	7 (t)
0	0	0	0	0	0	0	0	0
(s) 1	0	0	0	0	0	0	1	1
t 2	0	0	0	0	0	0	1	2
o 3	0	0	1	1	1	1	1	2
n 4	0	0	1	2	2	2	2	2
e 5	0	0	1	2	2	3	3	(3)

[Subsets {1, 2, 3} $2^3 = 8$]

*(Time Management)

Greedy Algorithms :→ (Coin Change)

* Minimum number of coins to reach a particular amount. (Duplicates allowed)

Coins = {1, 2, 5, 10, 20, 50, 100, 200, 500, 1000}

V = 91 = 50, 20, 20, 1   1 × 91

V = 31 = 20, 10, 1

3 = 2, 1

91 − (50) = 41
41 − (20) = 21
21 − (20) = 1
1 − (1) = 0

Important Greedy Algorithms & Questions For Placements :→

(i) Min Number of Coins
(ii) Min Cost of Connecting Ropes
(iii) Fractional Knapsack
(iv) 0-1 Knapsack
(v) Chocolate Distribution Problem
(vi) Huffman Encoding
(vii) Activity Selection Problem
(viii) Job Scheduling Problem
(ix) Policemen Catch Thieves
(x) Min No of Platforms
(xi) Gas Station Problem
(xii) Min arrows to burst all balloons
(xiii) Best time to buy or sell stock
(xiv) Nikunj & Donuts
(xv) Lemonade Stand
(xvi) Minimum no of jumps
(xvii) All sliding window & graph algos.

LC, GFG, Code forces

One last Question :→

Do you honestly think that this 15 days of training has improved your tech + coding skills by at least (10) − 20 % ?

10 mins ⇒

Yes / No

Practice
DRY RUN